

Reviewer Pack — Minimal Rank of Twisted Quotient Algebras vs DPLL Proof Leng...

Entry f71b55882d9c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-25 18:57:54 UTC

Minimal Rank of Twisted Quotient Algebras vs DPLL Proof Length

Entry ID: f71b55882d9c **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-25 18:57:54 UTC

1. Conjecture

Field A (mathematical branch): Twisted Module Theory **Field B** (complexity object): Complexity Theory: Resolution Proof Complexity

Statement:

{‘sentence_1’: ‘For a given satisfiability problem instance, the minimal rank of the twisted quotient algebra constructed from its clause-indicator polynomial modulo 2 is superlinearly related to the DPLL proof length.’, ‘sentence_2’: ‘Specifically, if we denote the minimal rank as ρ and the DPLL proof length as l , then for all instances with n variables, $\rho(\Phi) = \Omega(n^\alpha)$ for some $\alpha > 1$, where Φ is the clause-indicator polynomial modulo 2.’, ‘sentence_3’: ‘This relation holds for all instances of size $n \leq 40$ and 30 random seeds are used to ensure statistical robustness.’}

Rationale (proposer’s reasoning):

{‘sentence_1’: ‘Twisted module theory provides a framework that generalizes the representation theory of finite groups and has potential applications in algebraic complexity theory.’, ‘sentence_2’: ‘The proposed mapping from clause-indicator polynomials to twisted quotient algebras could potentially expose hidden structures related to the resolution proof complexity, leading to new insights into the hardness of SAT problems.’, ‘sentence_3’: ‘This conjecture aims to bridge the gap between representation theory and computational complexity by using algebraic tools to study the structure of resolution proofs.’}

Taxonomy category: GCT_DET_PERM (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): dc49770b97cd278d

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if for all instances with n variables ($n \leq 40$), the Spearman's rank correlation coefficient between minimal rank ρ and DPLL proof length l exceeds 0.7, calculated across 30 random seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.70	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "twisted quotient algebra" AND "resolution proof complexity" - "minimal rank" Twisted Module Theory AND DPLL proof length - "clause-indicator polynomial modulo 2" AND $\Omega(n^\alpha)$

Top relevant hits considered: - [http://arxiv.org/abs/2501.15003v2] Twisted intertwining operators and tensor products of (generalized) twisted modules - [http://arxiv.org/abs/1905.13138v3] Generators, spanning sets and existence of twisted modules for a grading-restricted vertex (super)algebra - [http://arxiv.org/abs/1009.2195v1] A note on (α, β) -higher derivations and their extensions to modules of quotients

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction
from itertools import combinations

def gaussian_elimination(matrix):
    rows, cols = len(matrix), len(matrix[0])
    for i in range(rows):
        max_row = i + max(range(i, rows), key=lambda r: abs(matrix[r][i]))
        matrix[i], matrix[max_row] = matrix[max_row], matrix[i]
        for j in range(i + 1, rows):
            factor = -matrix[j][i] / matrix[i][i]
            for k in range(cols):
                if i == k:
                    matrix[j][k] = 0
                else:
                    matrix[j][k] += factor * matrix[i][k]
    return matrix

def rank(matrix):
```

```

rows, cols = len(matrix), len(matrix[0])
rref = gaussian_elimination(matrix)
rank = sum(1 for row in rref if any(row))
return rank

def clause_indicator_polynomial(instance):
    n = len(instance)
    poly = [0] * (2 ** n)
    for assignment in range(2 ** n):
        valid = True
        for clause in instance:
            if all(poly[assignment ^ 1 << i] == 0 for i, var in enumerate(clause)):
                valid = False
                break
        if valid:
            poly[assignment] += 1
    return [x % 2 for x in poly]

def dpll(instance):
    def backtrack(assignment):
        if all(any(poly[assignment ^ 1 << i] == 0 for i, var in enumerate(clause)) for clause in instance):
            return True
        if any(all(poly[assignment ^ 1 << i] == 0 for i, var in enumerate(clause)) for clause in instance):
            return False
        var = next(i for i in range(n) if assignment & (1 << i) == 0)
        for value in [0, 1]:
            new_assignment = assignment ^ (1 << var)
            poly[var] += value
            if backtrack(new_assignment):
                return True
            poly[var] -= value
        return False

    n = len(instance)
    poly = [0] * n
    return backtrack(0)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    instance = [[random.randint(0, n - 1) for _ in range(random.randint(2, 3))] for _ in range(n)]

    poly = clause_indicator_polynomial(instance)
    proof_length = dpll(instance)

    if proof_length == 0:
        return {
            "metric_name": "Minimal Rank",
            "metric_value": 0,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "DPLL proof length is 0, cannot compute rank"
        }

    rho = rank([[poly[assignment ^ 1 << i] for i in range(n)] for assignment in range(2 ** n)])

```

```

    return {
        "metric_name": "Minimal Rank",
        "metric_value": rho,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else list(range(2, 30 * 1000 + 1, 100))

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_rho = sum(r["metric_value"] for r in results) / len(results)
    std_rho = math.sqrt(sum((r["metric_value"] - mean_rho) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if not r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_rho} std={std_rho} support_fraction={support_fraction}")
    elif any(r["counterexample"] != "" for r in results):
        first_failing_seed = next(s for s, r in enumerate(results) if r["counterexample"] != "")
        print(f"RESULT: FALSIFIED counterexample=\"{results[first_failing_seed]['counterexample']}\"")
    else:
        print("RESULT: INCONCLUSIVE insufficient support")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c775192a.py", line 102, in <module>
    result = run_trial(seed)
              ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c775192a.py", line 74, in run_trial
    poly = clause_indicator_polynomial(instance)
           ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c775192a.py", line 39, in clause_indica
    poly = [0] * (2 ** n)
           ~~~^~~~~~
MemoryError

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed due to a MemoryError before producing data, which means the pre-registered support condition was not met and the critic challenged the result. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13037
2	preregistration	ollama_remote	glm4:latest	0	0	5773
3	novelty	ollama_remote	glm4:latest	0	0	4616
4	novelty	ollama_remote	glm4:latest	0	0	5579
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	45708
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	31194
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17226
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12701
9	judge	ollama_remote	glm4:latest	0	0	55047

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 190882 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/f71b55882d9c.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/f71b55882d9c.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/f71b55882d9c.tar.gz (if generated)